
Transformar una aplicacion FSM a raft_rx

Versión para C

2026-05-20

Contents

Transformar una aplicación FSM en C a una aplicación distribuida tolerante a fallos con	
raft_rx	5
1. Punto de partida: una FSM local y determinista	5
2. Qué problema resuelve Raft	6
3. Enumerar el alfabeto de entradas	7
4. Convertir entradas de dominio en comandos Raft	8
5. Escribir el adaptador raft_application_t	9
6. Mantener la aplicación idempotente	10
7. Snapshot y recuperación	11
8. Ejecutar Raft y la CLI dentro de rxnet	12
9. La CLI como otra FSM	13
10. Transporte TCP, peers y membresía	14
11. Transiciones automáticas y tiempo local	15
12. Commit-reveal como ejemplo de determinismo	16
13. Pruebas por niveles	16
13.1. Pruebas de comandos y hash	16
13.2. Pruebas de FSM pura	17
13.3. Pruebas del adaptador Raft	17
13.4. Pruebas de proceso real	17
14. Checklist general para transformar otra FSM en C	17
15. Errores comunes	18
16. Resultado buscado	18

List of Figures

Transformar una aplicación FSM en C a una aplicación distribuida tolerante a fallos con `raft_rx`

Esta guía explica, paso a paso, cómo pasar de una aplicación local basada en máquinas de estados a una aplicación distribuida, replicada y tolerante a fallos con Raft. Usa como ejemplo esta versión C de piedra-papel-tijera (`rps_c`), que se apoya en dos bibliotecas externas:

- `rxnet/c`: runtime cooperativo y máquinas de estados.
- `raft_rx/c`: consenso Raft, log replicado, transporte y persistencia.

El objetivo no es esconder Raft detrás de una API grande. El objetivo es que la arquitectura siga siendo visible y verificable: una FSM pura de dominio, una capa pequeña que convierte entradas en comandos Raft, una aplicación Raft que aplica comandos comprometidos, y un `main` que solo cablea componentes.

1. Punto de partida: una FSM local y determinista

Antes de distribuir una aplicación hay que tener claro qué se está distribuyendo. En este proyecto, el núcleo es la máquina de estados del juego, implementada en:

```
1 include/rps/rps_game.h
2 src/rps_game.c
```

La FSM tiene estados finitos:

```
1 typedef enum {
2     RPS_STATE_WAITING_PLAYERS = 0,
3     RPS_STATE_CONFIGURING_GAME = 1,
4     RPS_STATE_ROUND_OPEN = 2,
5     RPS_STATE_WAITING_COMMITS = 3,
6     RPS_STATE_WAITING_REVEALS = 4,
7     RPS_STATE_RESOLVING_ROUND = 5,
8     RPS_STATE_ROUND_FINISHED = 6,
9     RPS_STATE_GAME_OVER = 7
```

```
10 } rps_state_t;
```

También tiene entradas explícitas:

- añadir jugador;
- confirmar jugadores;
- configurar partida;
- abrir ronda;
- registrar commit;
- revelar jugada;
- avanzar el reloj lógico de la partida.

La idea fundamental es que la FSM no sabe nada de sockets, líderes, mayorías, logs, clientes remotos ni reintentos. Sus funciones públicas describen el dominio:

```
1 int rps_game_add_player(rps_game_t *game, const char *player_id);
2 int rps_game_confirm_players(rps_game_t *game);
3 int rps_game_configure(rps_game_t *game, const rps_game_config_t *
    config);
4 int rps_game_start_round(rps_game_t *game);
5 int rps_game_submit_commit(rps_game_t *game, const char *player_id,
6                             const char *commit_hash);
7 int rps_game_reveal(rps_game_t *game, const char *player_id,
8                     rps_move_t move, const char *nonce);
9 void rps_game_tick(rps_game_t *game, int count);
```

Esta separación es importante. Una FSM local bien definida se puede probar, trazar y razonar antes de introducir distribución. Si una aplicación local ya mezcla reglas de negocio con red y persistencia, Raft no la arregla: solo hará que los errores sean replicados de forma consistente.

2. Qué problema resuelve Raft

En una aplicación local, una llamada como esta muta inmediatamente el estado:

```
1 rps_game_submit_commit(&game, "alice", hash);
```

En una aplicación distribuida eso no es aceptable. Si cada nodo ejecuta acciones de usuario según el orden en que le llegan por TCP, dos nodos pueden observar órdenes distintos:

```
1 n1 ve: commit(alice), commit(bob)
2 n2 ve: commit(bob), commit(alice)
```

En piedra-papel-tijera puede parecer inofensivo, pero en cuanto hay timeouts, errores, configuración, rondas o puntuación, el orden importa. El problema no es solo propagar eventos; el problema es

acordar un único orden global.

Raft convierte varias copias de la aplicación en una máquina de estados replicada:

1. El cliente propone una acción al líder.
2. El líder la añade a su log.
3. El líder replica esa entrada a los seguidores.
4. Cuando una mayoría la guarda, la entrada queda comprometida.
5. Cada nodo aplica las entradas comprometidas en orden.
6. Todas las réplicas que aplican el mismo prefijo del log llegan al mismo estado.

La regla de oro es:

Una acción de usuario no existe para la FSM hasta que está comprometida en el log de Raft.

En C, esta regla debe verse en el código. La CLI no debe llamar directamente a `rps_game_*`. La CLI crea un `raft_command_t`. La FSM se modifica solo desde `rps_app.c`, dentro de la callback `apply()`.

3. Enumerar el alfabeto de entradas

El primer paso práctico para convertir una FSM local en una aplicación Raft es enumerar todas las entradas externas que pueden cambiar su estado.

En esta aplicación son:

```
1 typedef enum {
2     RPS_OP_UNKNOWN = 0,
3     RPS_OP_PLAYER_JOINED,
4     RPS_OP_CONFIRM_PLAYERS,
5     RPS_OP_CONFIGURE,
6     RPS_OP_START_ROUND,
7     RPS_OP_COMMIT_MOVE,
8     RPS_OP_REVEAL_MOVE,
9     RPS_OP_PEER_SET
10 } rps_operation_kind_t;
```

Estas operaciones viven en:

```
1 include/rps/rps_command.h
2 src/rps_command.c
```

Cada operación contiene solo datos serializables y acotados:


```

1 typedef struct {
2     rps_operation_kind_t kind;
3     char operation_id[RAFT_MAX_KEY];
4     char player_id[RPS_MAX_ID];
5     rps_game_config_t config;
6     char commit_hash[RPS_HASH_HEX];
7     rps_move_t move;
8     char nonce[RPS_MAX_TEXT];
9     int count;
10    char peer_value[RAFT_MAX_VALUE];
11 } rps_operation_t;

```

En C conviene ser disciplinado con tamaños máximos. `raft_rx` usa estructuras fijas como `raft_command_t`, con campos limitados:

```

1 typedef struct {
2     char op[32];
3     char key[RAFT_MAX_KEY];
4     char value[RAFT_MAX_VALUE];
5 } raft_command_t;

```

Eso obliga a diseñar comandos compactos. La ventaja es que la memoria y los límites son explícitos. La desventaja es que no se puede meter cualquier JSON grande sin pensar en límites, truncamientos y compatibilidad.

4. Convertir entradas de dominio en comandos Raft

La operación de dominio se codifica en un `raft_command_t`:

```

1 int rps_command_encode(const rps_operation_t *operation,
2                        int sequence,
3                        raft_command_t *command);

```

Por ejemplo, una operación de commit se convierte en:

```

1 op    = "rps.commit_move"
2 key   = "00000007"
3 value = "alice,4
          d936656be1127ebc333b0cc1fa36668cda69b303bc21501127c7196ff670333"

```

El campo `op` tiene prefijo `rps` . para separar comandos del juego de otros comandos de infraestructura. En esta implementación también existe `rps.peer_set`, que permite que la aplicación comprometida aprenda direcciones de peers cuando sea necesario.

La función inversa es:

```
1 int rps_command_decode(const raft_command_t *command,
2                       rps_operation_t *operation);
```

La regla práctica es simple:

- la CLI y los clientes producen comandos;
- Raft replica comandos;
- la aplicación Raft decodifica comandos;
- la FSM consume operaciones ya comprometidas.

Si una función de UI modifica directamente `rps_game_t`, se ha roto el diseño.

5. Escribir el adaptador `raft_application_t`

`raft_rx` no conoce las reglas del juego. Solo conoce una interfaz de aplicación:

```
1 typedef struct {
2     void (*apply)(void *user, const raft_command_t *command);
3     void (*reload)(void *user);
4     size_t (*snapshot)(void *user, void *buf, size_t buf_size);
5     void (*restore_snapshot)(void *user, const void *buf, size_t size);
6     void *user;
7 } raft_application_t;
```

La implementación de RPS está en:

```
1 include/rps/rps_app.h
2 src/rps_app.c
```

Su estado principal es:

```
1 typedef struct {
2     rps_game_t game;
3     char applied_keys[RAFT_MAX_LOG][RAFT_MAX_KEY];
4     size_t applied_key_count;
5     char applied_operation_ids[RAFT_MAX_LOG][RAFT_MAX_KEY];
6     size_t applied_operation_id_count;
7     raft_command_t applied_commands[RAFT_MAX_LOG];
8     size_t applied_command_count;
9     rps_peer_set_fn peer_set;
10    void *peer_set_user;
11 } rps_app_t;
```

El método central es `apply()`. Su forma conceptual es:

```

1 static void app_apply(void *user, const raft_command_t *command) {
2     rps_app_t *state = (rps_app_t *)user;
3     rps_operation_t op;
4
5     if (command_key_already_applied(state, command->key)) {
6         return;
7     }
8
9     if (rps_command_decode(command, &op) != 0) {
10        return;
11    }
12
13    apply_operation(state, &op);
14    remember_applied_command(state, command);
15 }

```

Y `apply_operation()` traduce operaciones a entradas de la FSM:

```

1 case RPS_OP_PLAYER_JOINED:
2     rps_game_add_player(&state->game, op->player_id);
3     break;
4 case RPS_OP_CONFIGURE:
5     rps_game_configure(&state->game, &op->config);
6     break;
7 case RPS_OP_COMMIT_MOVE:
8     rps_game_submit_commit(&state->game, op->player_id, op->commit_hash
9     );
10    break;
11 case RPS_OP_REVEAL_MOVE:
12     rps_game_reveal(&state->game, op->player_id, op->move, op->nonce);
13     break;

```

Este adaptador debe ser aburrido. No debe contener la lógica de quién gana una ronda. Esa lógica pertenece a `rps_game.c`. El adaptador solo deserializa, comprueba duplicados y llama a la FSM.

6. Mantener la aplicación idempotente

Raft aplica cada entrada comprometida una vez y en orden, pero una aplicación robusta tiene que tolerar replay y duplicados:

- un nodo reiniciado puede reconstruir estado desde log o snapshot;
- un transporte puede entregar mensajes duplicados;
- un cliente puede reintentar una operación porque no recibió respuesta;
- un snapshot puede restaurarse y luego reaplicarse un sufijo del log.

Por eso `rps_app_t` guarda:

- `applied_keys`: claves de comandos ya aplicados;
- `applied_operation_ids`: identificadores lógicos de operaciones ya aplicadas;
- `applied_commands`: comandos aplicados, usados para snapshot/replay.

La distinción es importante:

- `command.key` identifica una entrada concreta del log.
- `operation_id` identifica la intención lógica de un cliente.

Ejemplo:

```
1 command.key    = "00000123"
2 operation_id   = "alice:commit:round-1"
```

Si el cliente reintenta la misma acción y el sistema acaba viendo otra entrada de log con otra `key`, `operation_id` permite convertirla en no-op. En esta primera versión C el campo existe en `rps_operation_t`; la CLI todavía usa una secuencia local simple. Para clientes reales conviene que el identificador lo genere el cliente y sea estable entre reintentos.

La idempotencia también debe existir en la FSM:

- añadir dos veces el mismo jugador debe ser no-op, no error;
- repetir el mismo commit del mismo jugador con el mismo hash debe ser no-op;
- repetir el mismo reveal del mismo jugador con la misma jugada debe ser no-op;
- repetir `start_round` justo después de abrir la ronda debe ser no-op;
- un commit repetido con otro hash sí es conflicto;
- un reveal con hash inválido no debe mutar `reveals`;
- replayar una operación ya aplicada no debe duplicar puntuación.

7. Snapshot y recuperación

Raft no puede crecer indefinidamente. En algún momento compacta el log y guarda un snapshot de la aplicación. Por eso `raft_application_t` exige:

```
1 size_t (*snapshot)(void *user, void *buf, size_t buf_size);
2 void (*restore_snapshot)(void *user, const void *buf, size_t size);
```

En esta versión, el snapshot de RPS es didáctico: serializa los comandos aplicados en un formato lineal compacto y `restore_snapshot()` reconstruye el estado llamando otra vez a `app_apply()`.

Conceptualmente:

```
1 rps.player_joined    00000001    alice
```

```

2 rps.player_joined      000000002    bob
3 rps.confirm_players    000000003
4 rps.configure          000000004    1,9,-1,-1
5 ...

```

No es el snapshot más pequeño posible, pero tiene dos ventajas docentes:

1. muestra que el estado de una FSM replicada puede reconstruirse desde una secuencia de entradas;
2. evita introducir dos formatos distintos, uno para comandos y otro para estado visible.

En una aplicación de producción se podría serializar directamente `rps_game_t` en un formato versionado. Si se hace eso, hay que incluir también cualquier metadato necesario para mantener idempotencia: claves aplicadas, operation ids procesadas, ronda actual, commits pendientes, reveals pendientes y puntuación.

8. Ejecutar Raft y la CLI dentro de rxnet

El ejecutable no debe ser un bucle manual lleno de lógica de negocio. En esta versión, `src/main.c` sigue el patrón del ejemplo de `raft_rx`:

```

1 typedef struct {
2     rx_fsm_runtime runtime;
3     raft_cluster_t cluster;
4     raft_tcp_transport_t tcp;
5     rx_coop_exec coop;
6     rx_fsm_machine cli_machine;
7     raft_node_t *node;
8     rps_app_t app_state;
9     rps_cli_t cli;
10 } rps_process_t;

```

El arranque hace composición:

1. inicializa el transporte TCP;
2. inicializa el runtime FSM;
3. inicializa el cluster Raft;
4. crea la aplicación RPS;
5. añade un nodo Raft al cluster;
6. conecta el transporte al nodo;
7. crea la FSM de CLI;
8. añade todo al ejecutivo cooperativo;
9. llama a `rx_coop_exec_run()`.

La parte importante de `main` es esta:

```
1 rx_fsm_runtime_init(&proc->runtime, 2);
2 raft_cluster_init(&proc->cluster, &proc->runtime);
3 raft_cluster_enable_realtime_clock(&proc->cluster);
4
5 app = rps_app_make_with_peers(&proc->app_state, register_peer_from_app,
6                               proc);
7 proc->node = raft_cluster_add_node(&proc->cluster, &cfg, args->data_dir
8                                   ,
9                                   &app, RPS_TICK_US);
10 proc->node->transport = raft_tcp_transport_make(&proc->tcp);
11
12 rps_cli_machine_init(&proc->cli_machine, &proc->cli);
13 rx_fsm_runtime_add_machine(&proc->runtime, &proc->cli_machine,
14                             RPS_TICK_US, 0);
15 rx_coop_exec_init(&proc->coop);
16 rx_coop_exec_add(&proc->coop, &proc->runtime.runtime);
17 rx_coop_exec_run(&proc->coop);
```

Observa que `main` no decide quién gana, no interpreta commits, no avanza rondas y no resuelve timeouts. Solo conecta piezas.

9. La CLI como otra FSM

La CLI está en:

```
1 include/rps/rps_cli.h
2 src/rps_cli.c
```

No es un REPL que muta el estado directamente. Es una FSM que:

1. en `latch_inputs`, comprueba si hay una línea en stdin;
2. drena comandos reenviados por followers;
3. drena solicitudes de join TCP;
4. en la acción de transición, parsea la línea del usuario;
5. en `dump_outputs`, procesa efectos externos seguros: forwarding, join y parada del ejecutivo.

Cuando el usuario escribe:

```
1 commit piedra alice-nonce
```

la CLI hace:

1. parsea `piedra` como `RPS_MOVE_ROCK`;

2. calcula el commitment SHA-256;
3. crea `RPS_OP_COMMIT_MOVE`;
4. codifica un `raft_command_t`;
5. si es líder, llama a `raft_node_submit_command()`;
6. si es follower, intenta reenviar al líder con `raft_tcp_transport_forward_cmd()`.

Lo que no hace es:

```
1 rps_game_submit_commit(&cli->app->game, ...);
```

Esa línea sería incorrecta porque aplicaría localmente antes del commit.

10. Transporte TCP, peers y membresía

`raft_rx` ofrece `raft_tcp_transport_t` para ejecutar nodos reales por TCP. Cada proceso necesita:

- un `--id` único;
- un puerto local con `--port`;
- una lista inicial de miembros con `--member`;
- direcciones conocidas con `--peer`;
- opcionalmente `--join HOST:PORT` para un nodo que entra por un introducir.

Ejemplo de tres nodos:

```
1 ./build/rps_node --id n1 --player alice --port 9101 \
2   --member n1 --member n2 --member n3 \
3   --peer n2=127.0.0.1:9102 --peer n3=127.0.0.1:9103
4
5 ./build/rps_node --id n2 --player bob --port 9102 \
6   --member n1 --member n2 --member n3 \
7   --peer n1=127.0.0.1:9101 --peer n3=127.0.0.1:9103
8
9 ./build/rps_node --id n3 --player carol --port 9103 \
10  --member n1 --member n2 --member n3 \
11  --peer n1=127.0.0.1:9101 --peer n2=127.0.0.1:9102
```

El transporte TCP no es la FSM de dominio. Es infraestructura. La aplicación puede registrar peers a través de un callback:

```
1 typedef void (*rps_peer_set_fn)(void *user,
2                                const char *node_id,
3                                const char *address);
```

En `main.c`, ese callback llama a:

```
1 raft_tcp_transport_add_peer(&proc->tcp, node_id, host, port);
```

Así, el conocimiento de direcciones se mantiene separado de las reglas del juego.

11. Transiciones automáticas y tiempo local

No todas las transiciones de la FSM son acciones de usuario. Algunas son consecuencia directa de haber llegado a un estado:

```
1 WAITING_COMMITS -- todos los commits recibidos --> WAITING_REVEALS
2 WAITING_REVEALS -- todos los reveals recibidos --> RESOLVING_ROUND
3 RESOLVING_ROUND -- resultado calculado --> ROUND_FINISHED
4 ROUND_FINISHED -- ganador o siguiente ronda --> GAME_OVER / ROUND_OPEN
```

Estas transiciones no necesitan una operación Raft propia. Después de aplicar una entrada comprometida, `rps_app.c` llama a una pequeña rutina local que avanza la FSM hasta que no quedan transiciones automáticas pendientes:

```
1 static void drain_automatic_transitions(rps_game_t *game) {
2     int i;
3     for (i = 0; i < 8; ++i) {
4         rps_state_t before = rps_game_state(game);
5         rps_game_tick(game, 1);
6         if (rps_game_state(game) == before) return;
7     }
8 }
```

Así, el log Raft contiene decisiones de juego, no pulsos de reloj. El orden replicado sigue siendo el de las operaciones importantes:

```
1 player_joined
2 confirm_players
3 configure
4 start_round
5 commit_move
6 reveal_move
```

Los timeouts de commit/reveal quedan como política local de avance de la FSM. Para este juego no rompen la consistencia esencial: las decisiones compartidas siguen entrando por Raft en el mismo orden. Si una aplicación futura necesita que un timeout sea una decisión global exacta, entonces ese timeout sí debería modelarse como una operación explícita del log.

La regla práctica queda así: sincroniza por Raft las decisiones que forman la historia de la partida; deja como locales las transiciones derivadas que no introducen información nueva.

12. Commit-reveal como ejemplo de determinismo

Piedra-papel-tijera usa un esquema commit-reveal para que un jugador no pueda esperar a ver la jugada del otro antes de elegir.

Primero se envía un hash:

```
1 rps_make_commitment("alice", RPS_MOVE_ROCK, "alice-nonce", hash, sizeof(hash));
```

El payload lógico es:

```
1 alice:0:alice-nonce
```

La función calcula SHA-256 y devuelve hexadecimal. Ese hash se replica como:

```
1 rps.commit_move 00000007 alice,<hash>
```

Después, Alice revela:

```
1 rps.reveal_move 00000008 alice,0,alice-nonce
```

Al aplicar `reveal_move`, la FSM vuelve a calcular el hash y lo compara con el commit previo. Si no coincide, registra error y no añade reveal.

Lo importante para Raft es que la validación sea determinista:

- misma entrada de log;
- mismo estado previo;
- misma función SHA-256;
- mismo resultado en todos los nodos.

No debe depender de memoria no inicializada, reloj local, orden de iteración no determinista, datos externos ni llamadas de red.

13. Pruebas por niveles

La transformación debe probarse de abajo arriba.

13.1. Pruebas de comandos y hash

`tests/test_hash_command.c` comprueba:

- que `rps_make_commitment()` produce el mismo SHA-256 que Python;

- que `rps_command_encode()` y `rps_command_decode()` preservan campos.

Esto evita errores de compatibilidad en el formato del log.

13.2. Pruebas de FSM pura

`tests/test_game.c` comprueba:

- partida feliz hasta `GAME_OVER`;
- puntuación;
- reveal inválido;
- timeout de reveal.

Estas pruebas no arrancan Raft. Si fallan, el problema está en el dominio, no en consenso.

13.3. Pruebas del adaptador Raft

`tests/test_app.c` aplica comandos a través de `raft_application_t`. Comprueba que:

- los duplicados no crean jugadores repetidos;
- las operaciones comprometidas conducen la FSM;
- snapshot y restore reconstruyen el estado.

Estas pruebas son el puente entre FSM local y log replicado.

13.4. Pruebas de proceso real

Finalmente se arranca un cluster TCP con varios `rps_node`. Lo mínimo a verificar es:

- todos abren puerto;
- se elige líder;
- followers conocen al líder;
- comandos enviados a followers llegan al líder;
- el commit index avanza;
- todos observan el mismo estado de juego.

14. Checklist general para transformar otra FSM en C

1. Extrae la FSM a un módulo sin dependencias de red ni Raft.

2. Define estados, entradas, guardas y acciones explícitas.
3. Añade tests de la FSM local antes de distribuirla.
4. Enumera todas las entradas externas que cambian estado.
5. Crea un tipo `operation_t` serializable y acotado.
6. Codifica cada operación como `raft_command_t`.
7. Implementa `decode(command, operation)`.
8. Escribe un `raft_application_t` fino: decode, idempotencia, llamada a FSM.
9. Asegúrate de que la FSM solo muta desde `apply()` en modo distribuido.
10. Añade deduplicación por `command.key` y, si hay clientes con reintentos, por `operation_id`.
11. Implementa `snapshot()` y `restore_snapshot()`.
12. Ejecuta Raft y CLI dentro de `rxnet`, no en bucles separados que muten estado compartido.
13. Convierte timeouts en entradas consensuadas o documenta claramente otro modelo temporal.
14. Mantén `main` como composición: argumentos, transporte, runtime, cluster, aplicación y ejecutivo.
15. Prueba por capas: comandos, FSM, adaptador, proceso real.

15. Errores comunes

- Aplicar localmente una acción de usuario antes de que Raft la comprometa.
- Poner reglas de dominio dentro de `app_apply()`.
- Usar el reloj local de cada nodo para transiciones que deben ser idénticas.
- Hacer que la CLI llame a `rps_game_*` directamente.
- No pensar en duplicados y replay.
- Serializar comandos sin límites claros de tamaño.
- Truncar strings en C sin comprobar que el formato sigue siendo válido.
- Guardar snapshots que no contienen suficiente información para reconstruir idempotencia.
- Mezclar descubrimiento de peers con reglas del juego.
- Convertir `main.c` en el lugar donde vive la lógica de aplicación.

16. Resultado buscado

La arquitectura final debe poder explicarse así:

```
1  usuario/CLI
2  |
3  v
```

```
4  rps_operation_t
5      |
6      v
7  raft_command_t
8      |
9      v
10 Raft leader -> replicated log -> committed entry
11     |
12     v
13 raft_application_t.apply()
14     |
15     v
16 rps_game_t FSM
17     |
18     v
19 estado visible: jugadores, ronda, commits, reveals, puntuación, ganador
```

Si todos los nodos aplican el mismo prefijo del log, todos observan la misma partida. Esa es la esencia de transformar una aplicación basada en máquinas de estados en una aplicación distribuida tolerante a fallos con Raft.